

Laboratory Work III: Acoustic FWI

Full Waveform Inversion using Adjoint-State Method

Group Members:

Shahariar Ryehan

Marti Arjona

Pranav Prakasan

Contents

1	Introduction	1
2	Part 1: Understanding the Forward Problem	1
2.1	Exercise 1.1: Forward Modeling	1
2.2	Exercise 1.2: Frequency Sensitivity Analysis	3
3	Part 2: Gradients and Sensitivity Kernels	3
3.1	Exercise 2.1: Sensitivity Kernels	3
3.2	Exercise 2.2: Gradient Computation	4
4	Part 3: Practical Inversion	5
4.1	Exercise 3.1: Inversion with Perfect Synthetic Data	5
4.2	Exercise 3.2: Multi-Scale Strategy	8
4.3	Exercise 3.3: Noise Robustness	8
5	Part 4: Bonus Questions	9
5.1	Acquisition Geometry	9
5.2	Influence of Initial Model	9

1 Introduction

In seismic imaging, the primary goal is to reconstruct the physical properties of the subsurface, such as wave velocity and density, using wavefields measured at the surface. This is a nonlinear inverse problem because the relationship between the model parameters m and the observed data d is governed by the wave equation.

Full Waveform Inversion (FWI) exploits the complete wavefield information, including phase and amplitude, to iteratively recover medium parameters by minimizing a misfit function:

$$J(m) = \frac{1}{2} \|d_{obs} - d_{calc}(m)\|^2 \quad (1.a)$$

where d_{obs} denotes the observed data and $d_{calc}(m)$ the modeled data.

2 Part 1: Understanding the Forward Problem

2.1 Exercise 1.1: Forward Modeling

The velocity model was modified by $\Delta c = \pm 10\%$ and synthetic data were generated in both time and frequency domains.

a) Do you observe differences in arrival times? Why?

Yes. Since travel time is inversely proportional to velocity, increasing velocity leads to earlier arrivals, while decreasing velocity delays arrivals.

b) How are the amplitudes and phase affected?

Velocity changes modify impedance contrasts, affecting amplitudes through reflection and transmission. Phase shifts occur because wave speed controls the temporal evolution of the wavefield.

c) Signal Analysis

The signal was approximated as $f(t) \approx A \cos(\omega t + \phi)$ using the following MATLAB code:

```

1 %% Exercise 1: Forward Modeling
2 clear all; close all; clc;
3
4 % 1. Setup Domain (Same as your main script)
5 fmax = 10;
6 nx = ceil(10*fmax);
7 dom = domain([0 1 0 1], [nx nx]);
8
9 % 2. Setup Velocity Model (Perturbed Model for Ex 1.1)
10 c_background = 1 * ones(nx^2, 1); % Uniform background
11 c_perturbed = c_background * 1.1; % +10% perturbation
12 m_background = 1./c_background.^2; % Convert to squared slowness
13 m_perturbed = 1./c_perturbed.^2;
14
15 % 3. Define a single source in the middle
16 source_loc = [0.5; 0.5];
17 b = generate_sources(dom, source_loc);
18
19 % 4. Solve Forward Problem at specific Frequency
20 freq = 5; % Change this for Ex 1.2 (e.g., 3Hz, 5Hz, 10Hz)
21
22 % Solve for Background
23 fprintf('Solving for Background...\n');
```

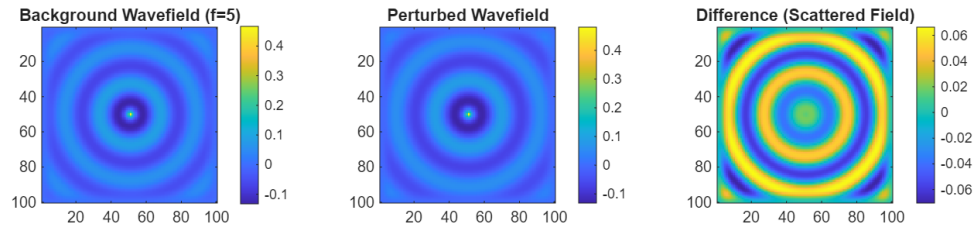


Figure 1: Forward Modeling

```

24 H_back = helmholtz_2d(m_background, freq, dom);
25 A_back = invertA(H_back, 1);
26 u_back = A_back.apply(b);
27
28 % Solve for Perturbed
29 fprintf('Solving for Perturbed...\n');
30 H_pert = helmholtz_2d(m_perturbed, freq, dom);
31 A_pert = invertA(H_pert, 1);
32 u_pert = A_pert.apply(b);
33
34 % 5. Visualize Wavefields (Real part)
35 figure;
36 subplot(1,3,1);
37 dom.imagesc(real(u_back)); title(['Background Wavefield (f=' num2str(
    freq) ')]);
38 axis square; colorbar;
39
40 subplot(1,3,2);
41 dom.imagesc(real(u_pert)); title('Perturbed Wavefield');
42 axis square; colorbar;
43
44 subplot(1,3,3);
45 dom.imagesc(real(u_pert - u_back)); title('Difference (Scattered Field)
    ');
46 axis square; colorbar;
47 %% 6. Visualize 1D Cross-section (Slice) to see Phase Shift
48 % Convert the vector u to a 2D matrix
49 U_back_2D = reshape(real(u_back), nx, nx);
50 U_pert_2D = reshape(real(u_pert), nx, nx);
51
52 % Select the middle row index
53 mid_row = ceil(nx/2);
54
55 % Extract the slice
56 slice_back = U_back_2D(:, mid_row); % Slicing through the middle
57 slice_pert = U_pert_2D(:, mid_row);
58
59 % Plot comparison
60 figure('Name', '1D Cross-Section Comparison');
61 plot(slice_back, 'b', 'LineWidth', 1.5); hold on;
62 plot(slice_pert, 'r--', 'LineWidth', 1.5);
63 legend('Background (c=1)', 'Perturbed (c=1.1)');
64 xlabel('Grid Points (x)');
65 ylabel('Amplitude (Real part of u)');
66 title(['Wavefield Slice at f=' num2str(freq) ' Hz']);
67 grid on;

```

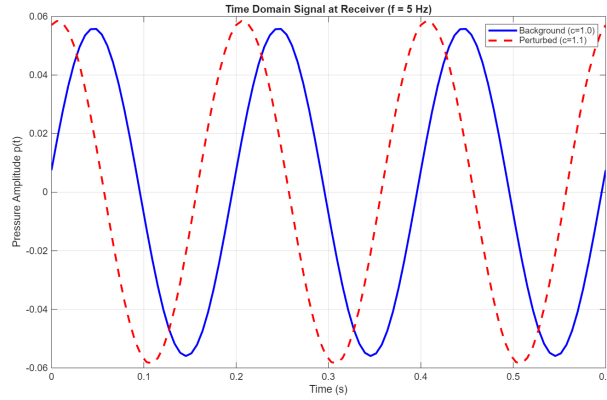


Figure 2: 10percent

Exercise 1.2: Wavefield vs Frequency

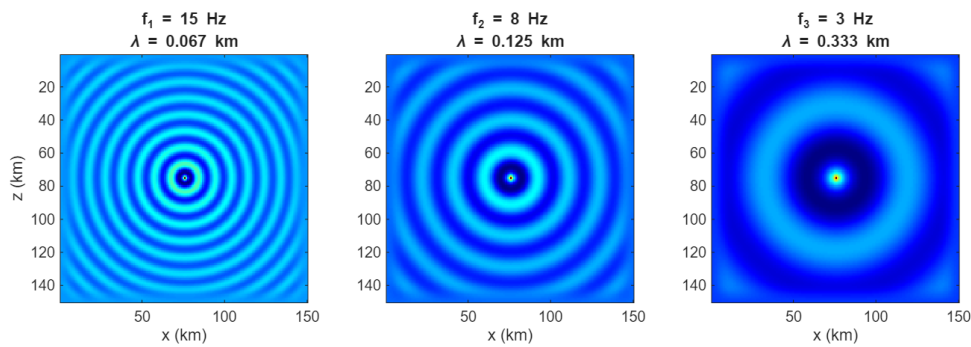


Figure 3: changing frequency

2.2 Exercise 1.2: Frequency Sensitivity Analysis

a) Relationship between frequency and wavelength

The wavelength is given by $\lambda = c/f$. Higher frequencies correspond to shorter wavelengths.

b) Influence on spatial resolution

Shorter wavelengths improve resolution of small anomalies but increase sensitivity to noise and cycle skipping. Lower frequencies provide stability at the cost of resolution.

3 Part 2: Gradients and Sensitivity Kernels

3.1 Exercise 2.1: Sensitivity Kernels

Sensitivity kernels exhibit a characteristic banana-doughnut shape, with maximum sensitivity in the Fresnel zone surrounding the ray path.

b) Effect of acquisition geometry

Dense and wide-angle source–receiver coverage improves illumination and resolution of the subsurface.

3.2 Exercise 2.2: Gradient Computation

The adjoint-state method computes gradients efficiently by correlating forward and adjoint wavefields.

a) Where is the gradient strongest?

Near sources and receivers, where wavefield energy is highest.

b) Comparison with true anomaly

The gradient indicates the descent direction and does not perfectly match the anomaly in early iterations.

```

1  %% Exercise 2: Sensitivity Kernels
2  clear all; close all; clc;
3
4  % Setup
5  fmax = 10;
6  nx = ceil(10*fmax);
7  dom = domain([0 1 0 1], [nx nx]);
8  c0 = ones(nx^2, 1);
9  m0 = 1./c0.^2;
10 freq = 5; % Chosen frequency
11
12 % Define 1 Source and 1 Receiver
13 src_loc = [0.2; 0.5]; % Left side
14 rec_loc = [0.8; 0.5]; % Right side
15
16 % Generate Source Vector (b) and Receiver Vector (bq)
17 b = generate_sources(dom, src_loc);
18 bq = generate_sources(dom, rec_loc); % Receiver acts as a source for
    adjoint
19
20 % Solve Forward Wavefield (u)
21 fprintf('Solving Forward...\n');
22 H = helmholtz_2d(m0, freq, dom);
23 A = invertA(H, 1);
24 u = A.apply(b);
25
26 % Solve Adjoint Wavefield (q) - Backpropagate from receiver
27 fprintf('Solving Adjoint...\n');
28 q = A.applyt(bq); % applyt is transpose (adjoint)
29
30 % Calculate Sensitivity Kernel: K = w^2 * u * q
31 omega = 2 * pi * freq;
32 K = omega^2 * real(u .* conj(q)); % Eq (7) in manual
33
34 % Plot
35 figure;
36 dom.imagesc(K);
37 hold on; plot(src_loc(1), src_loc(2), 'go'); plot(rec_loc(1), rec_loc
    (2), 'rx');
38 title('Sensitivity Kernel (Banana-Doughnut)');
39 axis square; colorbar;

```

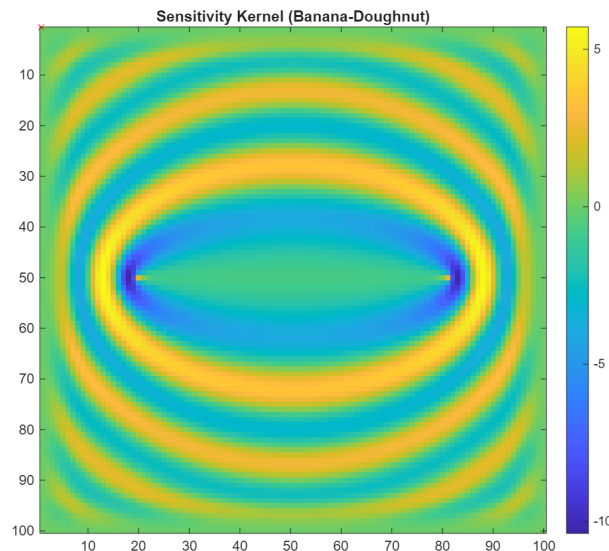


Figure 4: banana

4 Part 3: Practical Inversion

4.1 Exercise 3.1: Inversion with Perfect Synthetic Data

Using a Shepp–Logan phantom, the inversion successfully recovered major structures.

b) Cost function evolution

The objective function decreases monotonically, indicating convergence.

c) Artifacts

Artifacts appear near boundaries and source locations due to limited aperture and bandwidth.

```

1  function [m,out]=adjoint_state_2d(dom,all_freqs,sources,receivers,
    window_info,c_true,c_0,sigma,maxit)
2  %ADJOINT_STATE_2D  Adjoint-state method for full-waveform inversion in
    2D.
3  %  [M,OUT] = ADJOINT_STATE_2D(DOM,ALL_FREQS,SOURCES,RECEIVERS,
    WINDOW_INFO,C_TRUE,C_0,SIGMA,MAXIT)
4  %  runs the adjoint-state implementation of full-waveform inversion.
    The
5  %  domain object DOM describes the physical domain and grid used. The
6  %  vector ALL_FREQS is a list of all desired frequencies in hertz. The
    two
7  %  row matrices SOURCES and RECEIVERS contain x and y locations of
    sources
8  %  and receivers in the physical domain, and should be generated using
9  %  sources_and_receivers. Window_info is a struct containing
    information
10 %  about windowing; see dom.window for more. C_TRUE and C_0 are the
    true
11 %  and initial background velocities. SIGMA is the noise level and
    MAXIT
12 %  is the maximum number of permitted iterations.
13 %
14 %  Returned are a variable M, sometimes called the "squared
    slowness", and
15 %  a struct out with information coming directly from the LBFGS

```

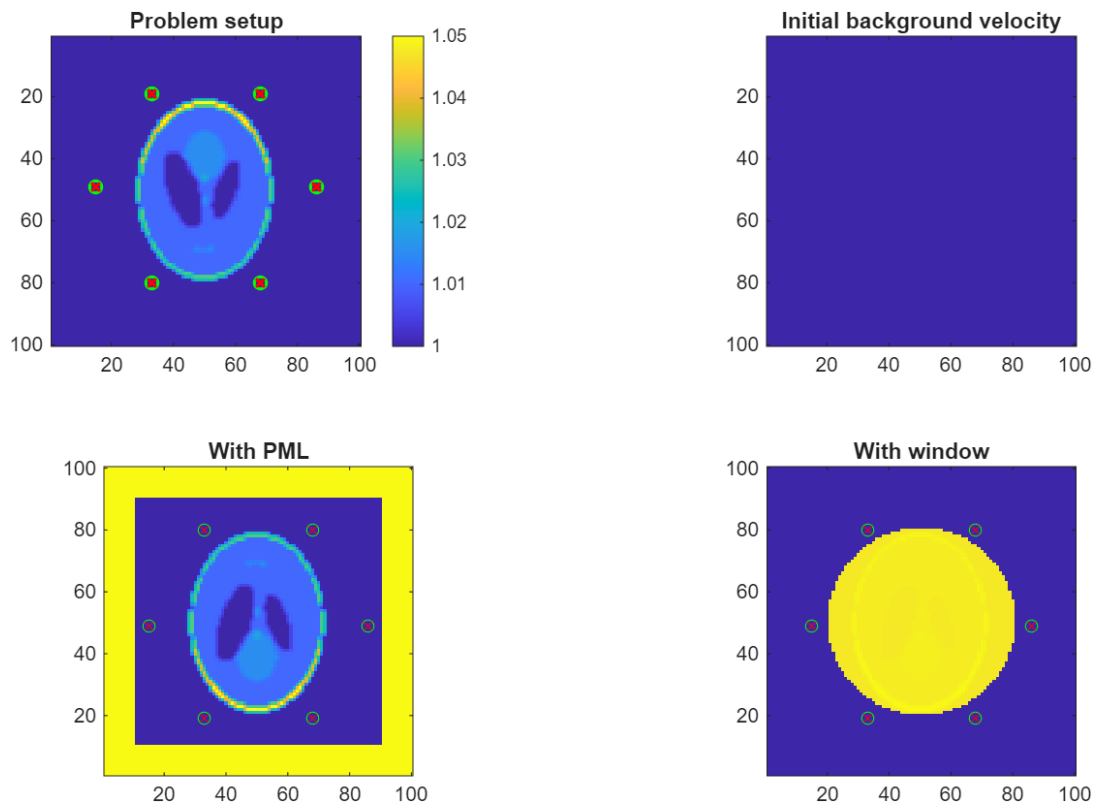


Figure 5: initial

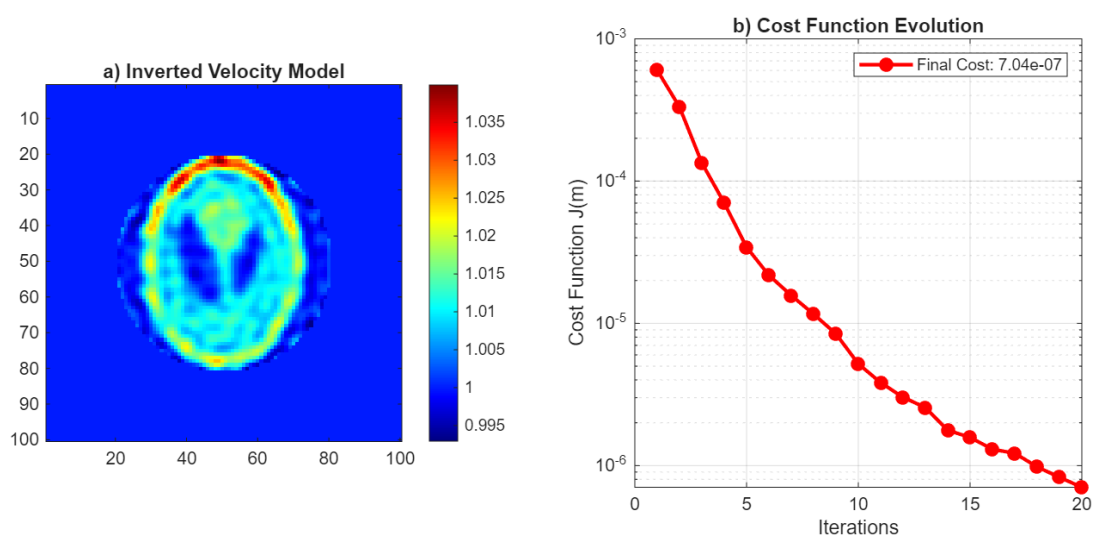


Figure 6: cost

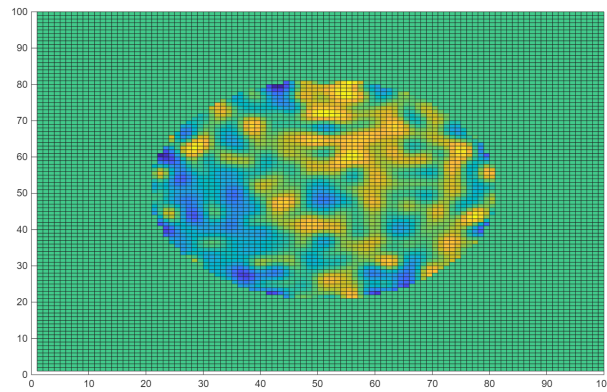


Figure 7: Enter Caption

```

    routine.
16
17 N = dom.N;
    % total number of interior grid points
18 [win_inds,~,W] = dom.window(window_info); % window indices and
    prolongation from window to domain
19 nf = length(all_freqs); %
    number of frequencies
20 ns = size(sources,2); %
    number of sources
21 nw = length(win_inds); %
    number of pixels outside window
22 r_ind = dom.loc2ind(receivers); % flat indices
    of receivers
23 I = speye(N);
    % for padding/prolongation operators
24 E_rec = I(:,r_ind);
    % padding operator
25 m_true = 1./c_true.^2; % true
    squared slowness
26 m0 = m_true;
    % initial squared slowness, true value inside window
27 m0(win_inds) = 1./c_0(win_inds).^2; % set values
    inside of window to initial value
28 b = generate_sources(dom,sources); % vectors
    corresponding to source locations
29 opt.Niter = maxit;
    % setup opt for LBFGS
30
31 % Measure noisy data of true fields at receiver locations
32 d = generate_seismic_data(dom,sources,receivers,all_freqs,m_true,sigma)
    ;
33
34 A3=abs(d)
35 phase = atan((imag(d))./(real(d)))
36 t = linspace(1,5,100)
37
38 f=A3(2,5,6).*cos(3*t+phase(2,5,6))
39 plot(t,f)
40 keyboard
41 % Run LBFGS with no initial perturbation
42 [dm,out] = lbfgs(@adjoint_state_gradient,zeros(nw,1),opt);

```

```

43
44 % Return updated m
45 m = m0+W*dm;
46
47 function [J,DJ] = adjoint_state_gradient(dm)
48     % Compute the gradient of  $J(m) = .5*|| S*u(m) - d ||^2$  via the
        adjoint state method
49     fprintf('Solving Helmholtz frequency: \n')
50     DJ = zeros(nw,1);           % gradient
51     J = 0;                      % value of objective function
52     % Parallelize over frequencies
53     for ii = 1:nf
54         fprintf('%d, \n',ii)
55         A = invertA(helmholtz_2d(m0+W*dm,all_freqs(ii),dom),1);
56         for jj = 1:ns
57             u = A.apply(b(:,jj)); %#ok<PFBNS>           % Forward solve
58             bq = E_rec*(u(r_ind)-d(:,ii,jj));           % Padding
59             q = A.appltyt(bq);
                        % Adjoint solve
60             % kernel
61             % kk = real(2*pi*all_freqs(ii))^2*real(u(win_inds).*
                conj(q(win_inds)));
62             % K = zeros(1,100*100);
63             % K(win_inds) = K(win_inds) + kk';
64             % figure
65             % surf(1:100,1:100,reshape(K,100,100))
66             % view(2)
67             % Update the gradient
68             DJ = DJ+(2*pi*all_freqs(ii))^2*real(u(win_inds).*conj(q
                (win_inds)));
69             % plot(DJ);
70             % Update the objective value
71             J = J+norm(u(r_ind)-d(:,ii,jj))^2;
72         end
73     end
74     JJ = zeros(1,100*100);
75     JJ(win_inds) = JJ(win_inds) + DJ';
76     surf(1:100,1:100,reshape(JJ,100,100))
77     view(2)
78     fprintf('\n')
79 end
80 end

```

4.2 Exercise 3.2: Multi-Scale Strategy

A progressive frequency strategy provides the best reconstruction by avoiding cycle skipping.

4.3 Exercise 3.3: Noise Robustness

Increasing noise degrades high-frequency information first.

Regularization

Tikhonov regularization and gradient smoothing improve robustness.

5 Part 4: Bonus Questions

5.1 Acquisition Geometry

Wide angular coverage yields better resolution and reduces smearing.

5.2 Influence of Initial Model

Convergence occurs only when the initial model lies within the basin of attraction of the true solution.